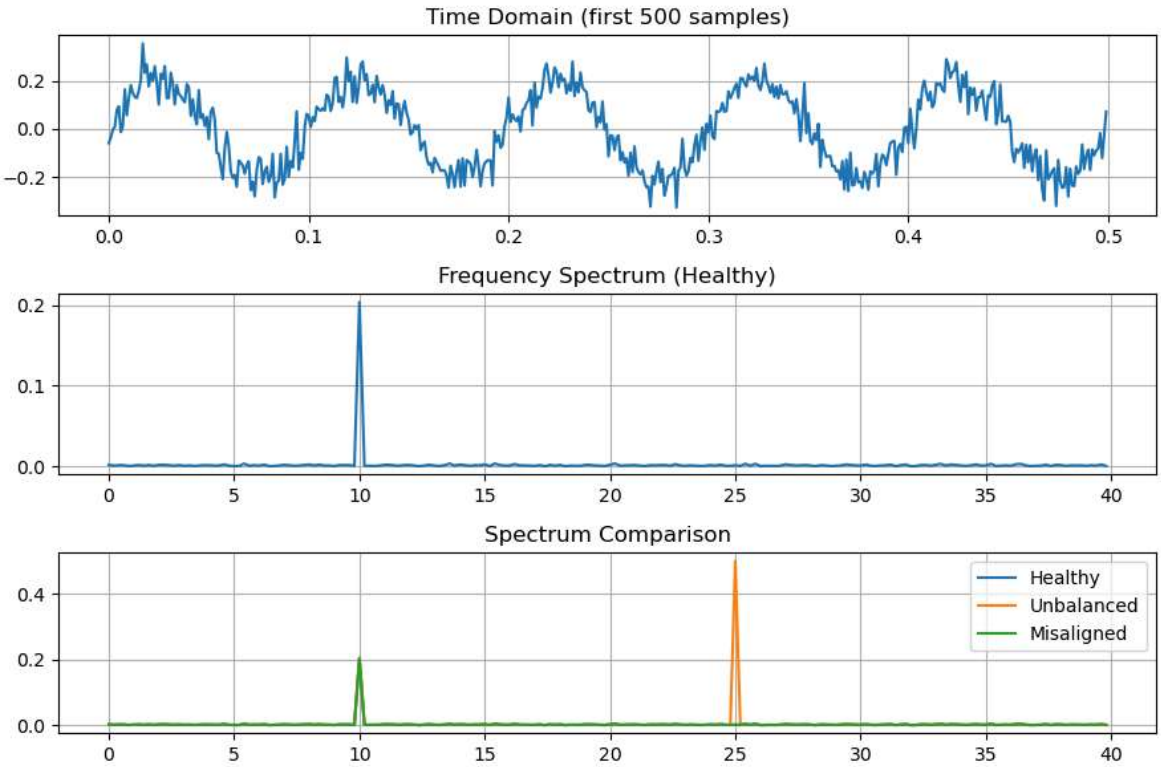
In [4]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.fft import fft, fftfreq

# --- Generate synthetic vibration data ---
fs = 1000          # Sampling frequency (Hz)
T = 5              # Duration (seconds)
t = np.arange(0, T, 1/fs)
N = len(t)

# Healthy signal: base vibration + noise
x = 0.2*np.sin(2*np.pi*10*t) + 0.05*np.random.randn(N)

# Save synthetic data to CSV (optional, for testing)
df = pd.DataFrame({"signal": x})
df.to_csv("vibrations.csv", index=False)

# --- Load data (works with synthetic or real CSV) ---
df = pd.read_csv("vibrations.csv")
print(df.head())

x = df["signal"].values

# --- FFT setup ---
f = fftfreq(N, 1/fs)[:N//2]
F = lambda s: 2/N * np.abs(fft(s)[:N//2])

# --- Fault simulations ---
u = x + 0.5*np.sin(2*np.pi*25*t)   # Unbalanced fault
m = x + 0.7*np.sin(2*np.pi*100*t)  # Misaligned fault

# --- Plots ---
plt.figure(figsize=(9,6))

# Time domain
plt.subplot(3,1,1)
plt.plot(t[:500], x[:500])
plt.title("Time Domain (first 500 samples)")
plt.grid()

# Frequency spectrum
plt.subplot(3,1,2)
plt.plot(f[:200], F(x)[:200])
plt.title("Frequency Spectrum (Healthy)")
plt.grid()

# Spectrum comparison
plt.subplot(3,1,3)
plt.plot(f[:200], F(x)[:200], label="Healthy")
plt.plot(f[:200], F(u)[:200], label="Unbalanced")
plt.plot(f[:200], F(m)[:200], label="Misaligned")
plt.legend()
plt.title("Spectrum Comparison")
plt.grid()

plt.tight_layout()
plt.show()
```

```
# --- RMS-based condition monitoring ---
rms = np.sqrt(np.mean(x**2))
cond = "HEALTHY" if rms < 0.5 else "UNBALANCED" if rms < 1 else "MISALIGNED / FA

print("\nVIBRATION ANALYSIS RESULT")
print("RMS Value:", round(rms,4))
print("Engine Condition:", cond)
```

```
     signal
0 -0.058924
1 -0.034088
2 -0.003197
3  0.007935
4  0.078589
```




```
VIBRATION ANALYSIS RESULT
RMS Value: 0.1525
Engine Condition: HEALTHY
```

In [ ]: